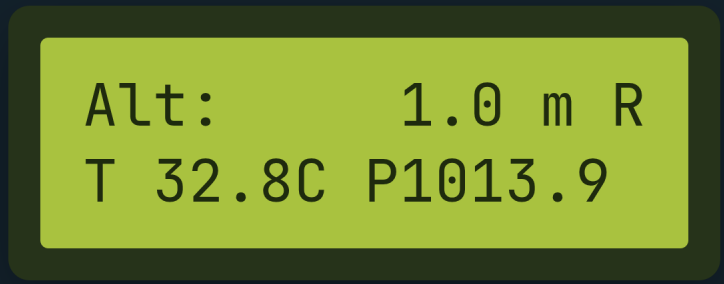
MPMC PROJECT · REVIEW 2

A barometric altimeter on the 8051

AT89S52 + BMP280 pressure sensor + 16×2 LCD, written in C and built entirely on Linux



Nithish Siva Kumar · 23BCE1230 · Guide: Dr. Sivasubramanian A

THE IDEA

A device that turns air pressure into height

01 · MEASURE

Read the air

A BMP280 sensor reports pressure and temperature over I2C.

02 · COMPUTE

Work out height

The AT89S52 converts pressure to altitude using integer maths only.

03 · SHOW

Display it

A 16×2 LCD shows altitude, temperature and pressure 4 times a second.

THE DEMO Tap the button to zero it, lift the board 1 m, and it reads about +1.0 m.

THE PHYSICS

Air pressure drops as you climb

Near sea level, you lose about

12 Pa

of pressure for every metre you climb. That’s 1 hPa for every 8.3 m.

A 1 m lift changes pressure by about 0.01 %, so the sensor has to resolve single pascals.

STANDARD ATMOSPHERE

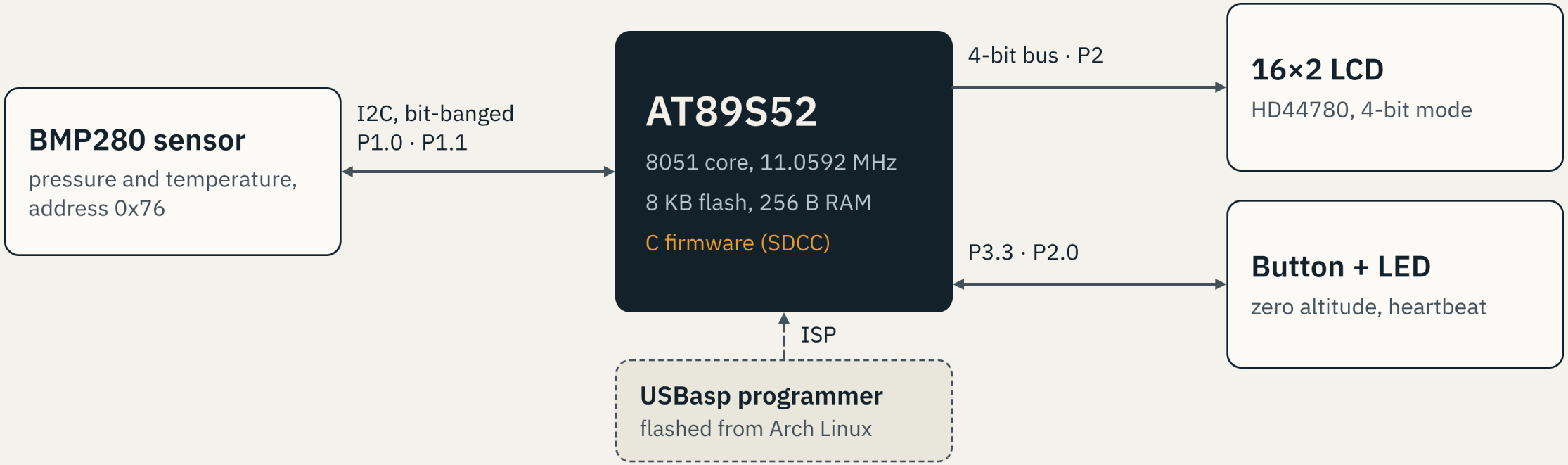
$$h = 44330 \times (1 - (P/P0)^{0.1903})$$

Absolute mode: P0 = 101 325 Pa, the standard sea-level pressure.

Relative mode: P0 = the pressure when you pressed the button. This is the one we demo.

BLOCK DIAGRAM

The system at a glance



HARDWARE

What’s on the bench

Part	What it is	Its job
AT89S52 dev board	8-bit 8051, 8 KB flash, 256 B RAM	Runs everything
BMP280 breakout	Bosch pressure and temperature sensor	Measures to ~1 Pa
16×2 LCD	HD44780 controller, 4-bit mode	Shows the readings
Button + LED	Onboard, on P3.3 and P2.0	Zero / heartbeat
USBasp programmer	USB ISP programmer, driven by avrdude	Loads the code
SDCC + avrdude	Open-source C compiler and flasher	Build on Linux

WIRING

Pin map

Signal	Pin	Notes
I2C SCL	P1.0	bit-banged clock
I2C SDA	P1.1	bit-banged data
LCD RS, E	P2.1, P2.2	control lines
LCD D4–D7	P2.4–P2.7	4-bit data, RW to GND
LCD contrast	P2.3	software PWM
Status LED	P2.0	blinks every reading
Zero button	P3.3	active low
UART TX / RX	P3.1 / P3.0	9600 baud via MAX232

DESIGN NOTES

Port 0 stays empty: on this board it’s open-drain with no pull-ups, so it can’t drive a line high.

LCD pins are written one bit at a time, so the LED and contrast pins on the same port are never disturbed.

FIRMWARE

How the program runs

AT POWER-UP

1
Contrast PWM
Timer2

2
UART
Timer1, 9600 baud

3
LCD init
4-bit sequence

4
I2C bus
free a stuck line

5
Sensor
find, reset, calibrate

6
Wait
200 ms for data

THEN, FOREVER

EVERY 50 MS

Check the button and count how long it’s held: tap = zero, 2 s hold = absolute mode.

EVERY 250 MS

Read sensor → compensate → smooth → altitude → redraw LCD and send UART line → toggle LED

THE SENSOR

Talking to the BMP280

Register	Value	What it does
0xD0 chip ID	0x58	BMP280
0xE0 reset	0xB6	soft reset, then wait 5 ms
0xF5 config	0x30	IIR filter ×16, 62.5 ms standby
0xF4 ctrl_meas	0x57	pressure ×16, temp ×2, normal mode
0x88–0x9F	read	24 bytes of factory calibration
0xF7–0xFC	read	raw 20-bit pressure and temperature

RAW → REAL

Each chip stores 12 factory constants. Bosch's 32-bit integer formulas combine them with the raw readings.

Out come temperature in 0.01 °C and pressure in whole pascals.

The sensor measures on its own about 9 times a second; we read the latest result.

PROBLEMS WE HIT

Four limits of the 8051 shaped every design decision.

No I2C hardware

8-bit ALU

No floating point

256 B of RAM

PROBLEM 1 OF 4

No I2C controller on the chip

THE CONSTRAINT

The AT89S52 has a UART, three timers and interrupts, but no I2C hardware. The BMP280 only speaks I2C or SPI.

WHAT WE DID

- Bit-banged I2C in C: software toggles P1.0 (SCL) and P1.1 (SDA) for START, address, ACK, data and STOP.
- Port 1 is quasi-bidirectional, so writing 1 only releases the line. The sensor can always pull it low.
- At start-up, up to 9 clock pulses free a sensor left mid-byte by a reset.

WHAT IT TAUGHT US I2C at the bit level: START, ACK, repeated START, NACK on the last byte.

PROBLEM 2 OF 4

An 8-bit CPU doing 32-bit maths

THE CONSTRAINT

The ALU is 8 bits wide and its only multiply is MUL AB, 8 × 8. In SDCC an int is just 16 bits, but Bosch's formulas need 32-bit intermediates, and 64-bit for full precision.

WHAT WE DID

- Used Bosch's 32-bit integer version, not the 64-bit one. 1 Pa resolution is about 8 cm of height.
- Cast every term to int32_t. Without it, 16-bit intermediates overflow silently.
- Read all 24 calibration bytes in one burst straight into a struct, since both sides are little-endian.

WHAT IT TAUGHT US Word size is the programmer's job: know where every bit of a result goes.

PROBLEM 3 OF 4

No floating point for the exponent

THE CONSTRAINT

Altitude needs (P/P0)^0.1903, a fractional power. With no FPU, powf runs in software through log and exp routines. It pushed flash to about 7.3 of 8 KB and overflowed RAM.

WHAT WE DID

- A Python script evaluates the formula every 1000 Pa from 50 000 to 110 000 Pa: 61 values kept in flash.
- The 8051 finds the two nearest points and interpolates linearly, with integer maths only.
- Error is under 0.1 m near sea level and 0.27 m at worst.

WHAT IT TAUGHT US Spend the memory you have (flash) to skip maths you can’t afford.

PROBLEM 4 OF 4

256 bytes of RAM for everything

THE CONSTRAINT

Only 128 bytes are directly addressable, shared by registers, globals and locals. SDCC gives each local a fixed slot, so our first build needed 125 bytes there and failed to link.

WHAT WE DID

- Moved calibration data and buffers to indirect RAM (__idata).
- Marked the 32-bit compensation functions __reentrant, so their locals live on the stack.
- One running average instead of an 8-sample array, and our own number printing instead of printf.

WHAT IT TAUGHT US Plan the memory map first: code, direct data, idata and stack.

ALONG THE WAY

Smaller problems we solved

LCD CONTRAST

No pot, had to use a timer.

With V0 grounded the screen was grey, and the resistors on hand blanked it. Timer2 now switches P2.3 at 3.6 kHz, 40 % duty, from a short assembly interrupt.

TOOLCHAIN

No Keil, no Windows.

SDCC compiles, avrdude flashes through the USBasp. We found avrdude’s 89S52 part name, wrote a udev rule.

OUR CHOICES

Easier routes we chose not to take

Limit	The easier route	Why we didn’t
No I2C hardware	An 8051 with built-in I2C, such as the Nuvoton N76E003 or Silicon Labs C8051F series	We’d learn to set a register, not how the protocol works
No floating point	An external maths coprocessor (Am9511 / Intel 8231A), or an MCU with an FPU	An extra chip that uses up Ports 0 and 2 as a bus, or leaving the 8051 behind
Tiny RAM	A bigger 8051 like the P89V51RD2: 1 KB RAM, 64 KB flash, same socket	Working inside the limit is the lesson
LCD contrast	A 10 kΩ potentiometer	None on hand, and PWM taught us timers and interrupts
Toolchain	Keil µVision on Windows, like the board’s owner	We wanted an open, scriptable Linux toolchain

WHY THE HARD WAY

We kept the constraints on purpose.

Real products are built on small chips because of cost, power and board space. Every workaround here is a skill that carries straight over.

01
Protocols at the bit level

02
Fixed-point and integer maths

03
Memory maps and linker errors

04
Timers and interrupts in assembly

RESULTS

Tested on the real board

Test	Result
I2C scan	sensor answers at 0x76
Chip ID	0x58, a BMP280
Temp and pressure	32.8 °C, 1013.9 hPa (warm room)
Absolute altitude	−5.5 m on a high-pressure day
1 m lift after zeroing	≈ +1.0 m, back to ≈ 0 when lowered
Button	tap zeroes, 2 s hold → absolute
LCD	readable head-on, no flicker
UART output	written, untested: no RS-232 adapter

78 %

of flash used

6402 of 8192 bytes

104 B

of stack left free

out of 256 B total RAM

LIMITATIONS

What it doesn’t do yet

Assumes 15 °C air

Warm air is thinner. At 32 °C, heights read about 5 % low.

Weather drift

0.1 hPa of weather change looks like 0.8 m. Re-zero every few hours.

Fixed sea-level pressure

Absolute mode assumes 1013.25 hPa. There’s no way to enter today’s value.

2–3 s to settle

The sensor filter and our running average trade speed for steadiness.

Zero lost at power-off

The AT89S52 has no EEPROM to remember the reference.

No sensor recovery

If the sensor drops out mid-run, the display freezes until reset.

FUTURE WORK

Where we’d take it next

01

Temperature-corrected height

Feed the measured temperature into the hypsometric equation.

02

Enter local pressure (QNH)

Set today’s sea-level pressure with the button for true absolute height.

03

Remember settings

Add a small AT24C02 EEPROM module on the same I2C bus to keep the zero after power-off.

04

Vertical speed

Show climb rate in m/s, with a buzzer that beeps on climb or sink.

05

Smarter filtering

A Kalman or alpha-beta filter to settle faster without more noise.

06

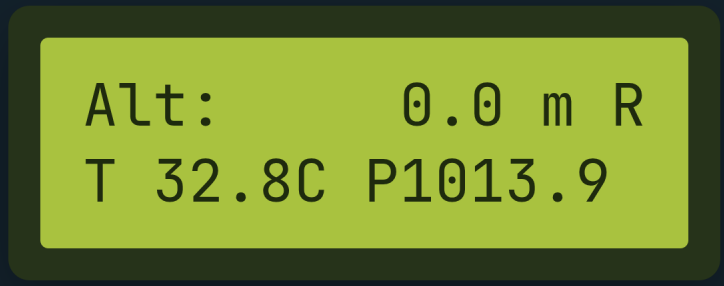
Logging to a PC

With a USB-serial adapter, stream readings and plot altitude live.

THANK YOU

Questions?

Live demo: tap to zero, lift the board 1 m, and watch it read +1.0 m.



Nithish Siva Kumar· 23BCE1230